

FinSight RAG: Multi-Agent RAG System for Financial News Investment Ideas

Ruochen Feng, Andrew Chen, Yikai Li

Final Project Report

Abstract

FinSight RAG is a multi-agent retrieval-augmented generation (RAG) system that converts financial news into structured short-term investment hypotheses. Rather than producing generic summaries, the system outputs a structured signal packet that includes direction, confidence, reasoning chain, risk factors, source citations, and a full agent execution trace. The current implementation includes a Streamlit dashboard, a news ingestion and RAG pipeline, technical factor and macro event enrichment, a three-agent reasoning workflow, and a forward-return evaluation module. This report includes implementation validation, a historical demo forward-return evaluation, and a RAG quality evaluation; a larger final live-data evaluation remains future work.

1. Introduction

Financial news contains useful market signals, but converting raw articles into actionable investment hypotheses is difficult. Headlines can be repetitive, public news may already be priced in, and simple sentiment analysis lacks the depth to weigh evidence or account for conflicting information. This project asks whether a multi-agent RAG pipeline can generate evidence-grounded, structured hypotheses that outperform simple sentiment and random baselines on short-horizon return prediction.

The project scope is limited to hypothesis generation—not trading execution or portfolio management. Each system output is a structured signal packet tied to specific source citations, making it both interpretable and testable. The signal schema and agent output include abstention and validation fields so low-evidence cases can be flagged rather than treated as high-confidence recommendations.

2. Depth of Research

2.1 Retrieval-Augmented Generation

Lewis et al. (2020) [1] introduced RAG as a method for grounding language model outputs in retrieved external documents. For financial applications, this grounding is essential: claims should be traceable to specific news sources, and output quality is tied to the relevance and credibility of retrieved evidence. FinSight RAG builds on this principle by adding source credibility weighting and citation-linked output validation to the standard retrieve-then-generate pipeline.

2.2 Multi-Agent Reasoning

Yao et al. (2022) [2] proposed ReAct, which interleaves reasoning steps with concrete actions such as retrieval and lookup. This decomposition motivates our three-agent design: an Analyst Agent that extracts evidence and key claims, a Strategist Agent that constructs a directional thesis with supporting and counter-evidence, and a Decision Agent that weighs evidence, assigns a confidence score, and produces the final signal packet. Separating these roles makes each step auditable and reduces the risk of conflating evidence extraction with hypothesis formation.

2.3 Financial NLP and Domain-Specific Evaluation

Wu et al. (2023) [3] (BloombergGPT) demonstrated that financial language has domain-specific patterns that require domain-adapted models and finance-specific evaluation benchmarks. This supports our decision to evaluate signal quality using directional hit rate against realized forward returns—a domain-relevant metric—rather than using fluency or coherence as proxies. The system's keyword/rule-based sentiment scoring is explicitly treated as a baseline, consistent with the finding that simple sentiment classifiers are insufficient for financial forecasting tasks.

Loughran and McDonald (2011) [5] developed a finance-specific sentiment dictionary that improves on general-purpose lexicons by accounting for terms that carry domain-specific meaning in financial text. This work informs the design of our keyword-based sentiment baseline, which uses domain-relevant bullish and bearish word lists. More sophisticated approaches such as FinBERT [6] are discussed in Section 6 as potential future improvements.

2.4 Why Simple Sentiment Is Insufficient

Sentiment classifiers capture valence but not novelty, source credibility, or whether information is already priced into the market. A positive headline does not imply a bullish forward return if the news is widely known or contradicted by other signals. FinSight RAG addresses this by explicitly extracting counter-evidence, weighting sources by credibility, enriching signals with technical and macro context, and evaluating outputs against realized returns rather than sentiment proxies.

2.5 Market Efficiency and Evaluation Design

Fama (1970) [4] formalized the Efficient Market Hypothesis (EMH), which holds that asset prices rapidly incorporate publicly available information. Under semi-strong efficiency, a signal

derived solely from public news should not systematically generate excess returns. This motivates rigorous evaluation: we compare against baselines, test on realized returns, and treat hypothesis generation as a research question rather than a guarantee of alpha.

3. Methodology

3.1 Experimental Setup

Each experiment unit is one signal generated for one ticker at one signal timestamp. Forward returns are attached after signal generation to avoid look-ahead bias. No future price data is available to any component at inference time.

- Tickers (current demo set): NVDA, TSLA, MSFT, AMD
- Prediction horizons: 5 trading days, 20 trading days
- Baselines: (1) random directional assignment; (2) keyword/rule-based sentiment direction
- Primary metrics: 5-day and 20-day directional hit rate, average signed return, average raw return, signal coverage rate, confidence-stratified hit rate
- RAG quality test set: AAPL, NVDA, TSLA, MSFT, GOOGL, AMZN, META, SPY, QQQ
- RAG quality metrics: Recall@K, Precision@K, MRR, context relevance, completeness, correctness, groundedness, coherence

3.2 Data Collection and Preprocessing

News ingestion is handled by three modules: `news_ingester.py` normalizes articles from Yahoo Finance and public RSS/Google News-style feeds; `bloomberg_api.py` provides optional integration with Bloomberg B-PIPE for high-authority metadata; `stock_screener.py` performs market activity scanning, event detection, and ticker universe filtering based on volume and price movement.

Each article is normalized into a common schema: citation ID, source name, headline, URL, timestamp (UTC-normalized), body text, associated ticker, and credibility weight. Preprocessing includes deduplication by URL and content hash and ticker relevance filtering. Source credibility weights are manually assigned as heuristic source-tier weights based on source type, reputation, and expected reliability. Articles are then chunked into RAGChunk objects for retrieval.

3.3 Retrieval

The retrieval layer (`rag_pipeline.py`) uses TF-IDF ranking with keyword fallback. Retrieved chunks are re-ranked by a credibility blend that upweights high-authority sources. The top-k RetrievedChunks are passed to the agent pipeline. All claims in the final output are required to be traceable to a specific retrieved chunk via citation ID. Dense vector retrieval (e.g., FAISS or

Chroma) is not currently implemented; it is identified as a future improvement that would likely improve semantic recall for paraphrase-heavy financial writing.

3.4 Evidence Enrichment

In addition to news and RAG chunks, each signal is enriched with two supplementary evidence layers:

- Technical factors (technical_factors.py): RSI, MACD, moving averages, Bollinger Bands, and momentum indicators derived from OHLCV price data.
- Macro events (macro_events.py): geopolitical context, central bank events, and macroeconomic releases relevant to the ticker and signal time.

This enrichment gives the Strategist Agent and Decision Agent market-context evidence beyond the news articles themselves.

3.5 System Architecture

Figure 1 shows the full FinSight RAG architecture, from external data sources through multi-agent reasoning to the output layer and dashboard.

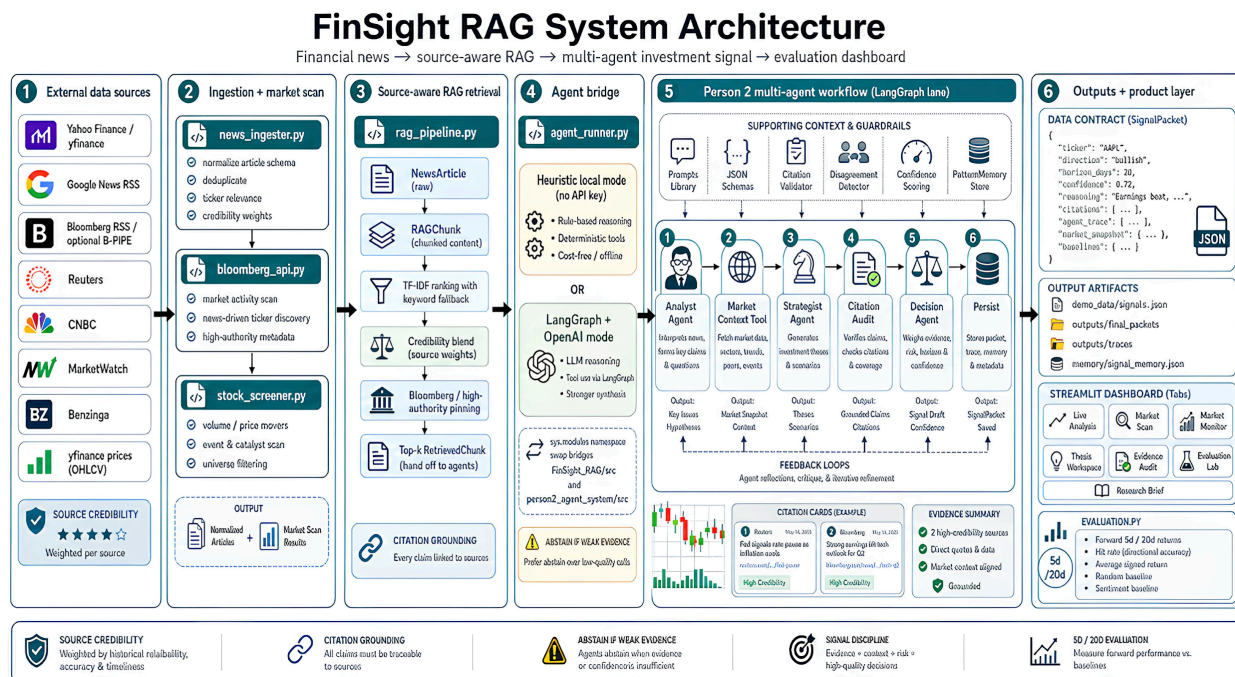


Figure 1. FinSight RAG System Architecture. News flows from external sources through source-aware RAG retrieval and a multi-agent reasoning workflow to a structured signal packet and Streamlit dashboard.

The pipeline has five layers:

- Ingestion: `news_ingester.py`, `bloomberg_api.py`, and `stock_screener.py` collect and normalize articles and market data.
- Retrieval: `rag_pipeline.py` chunks articles, ranks by TF-IDF and credibility blend, and returns top-k chunks to the agent bridge.
- Agent bridge (`agent_runner.py`): supports two execution modes — (1) heuristic/offline mode using keyword-based scoring and rule-based logic, no API key required; (2) LangGraph + LLM mode using GPT-4o-mini for full reasoning, if an OpenAI API key is provided.
- Multi-agent reasoning: three agents run in sequence. The Analyst Agent extracts key claims, events, and questions. The Strategist Agent constructs a directional thesis with causal chain, risks, and counter-evidence. The Decision Agent weighs evidence, assigns a calibrated confidence score, and emits the final signal packet. A disagreement detection module flags cases where agent assessments conflict.
- Output and evaluation: signal packets are written as JSON (`schemas.py`). The evaluation module (`evaluation.py`) attaches 5-day and 20-day forward returns and computes metrics against the random and keyword-sentiment baselines.

3.6 Signal Packet Schema

Table 1 lists all fields in the final signal packet output by the Decision Agent.

Field	Description
ticker / company / sector	Target asset identification and sector classification
benchmark	Reference index for relative performance context
event_type	Catalyst classification (earnings, macro, regulatory, etc.)
direction	Predicted direction: bullish, bearish, or neutral
horizon_days	Time horizon for the signal (5 or 20 trading days)
confidence	Calibrated confidence score (0–1)
novelty_score	Estimate of how new or unpriced the news event is
sentiment_score	Keyword/rule-based sentiment score using bullish/bearish word lists
source_quality	Aggregate heuristic credibility score of cited sources
published_at	Timestamp of the triggering article
reasoning	Natural language reasoning chain
catalyst	Primary event or factor driving the thesis
thesis_bullets	Structured thesis supporting points

risk_factors	Key risks that could invalidate the thesis
counter_evidence	Retrieved evidence that contradicts the thesis
watch_items	Conditions to monitor after signal issuance
market_snapshot	Market context at signal time (price, volume, technicals)
citations	Source citations for all claims (citation IDs)
agent_trace	Full multi-agent execution trace for auditability
baseline_sentiment / baseline_random	Keyword-sentiment and random baseline predictions for comparison

Table 1. Signal packet fields. The sentiment_score field uses keyword/rule-based scoring with bullish/bearish word lists, not a neural model.

3.7 Streamlit Dashboard

The Streamlit application (app.py) provides five tabs for interacting with the system:

- Live Analysis: submit a ticker and run the full pipeline in real time.
- Market Scan: view recent signals and market scan results across the ticker universe.
- Market Monitor: monitor live price and news activity.
- Evidence Audit: inspect retrieved evidence chunks, source credibility scores, and citation coverage for any signal.
- Evaluation / Backtesting: view forward-return metrics and baseline comparisons in the Evaluation Lab.

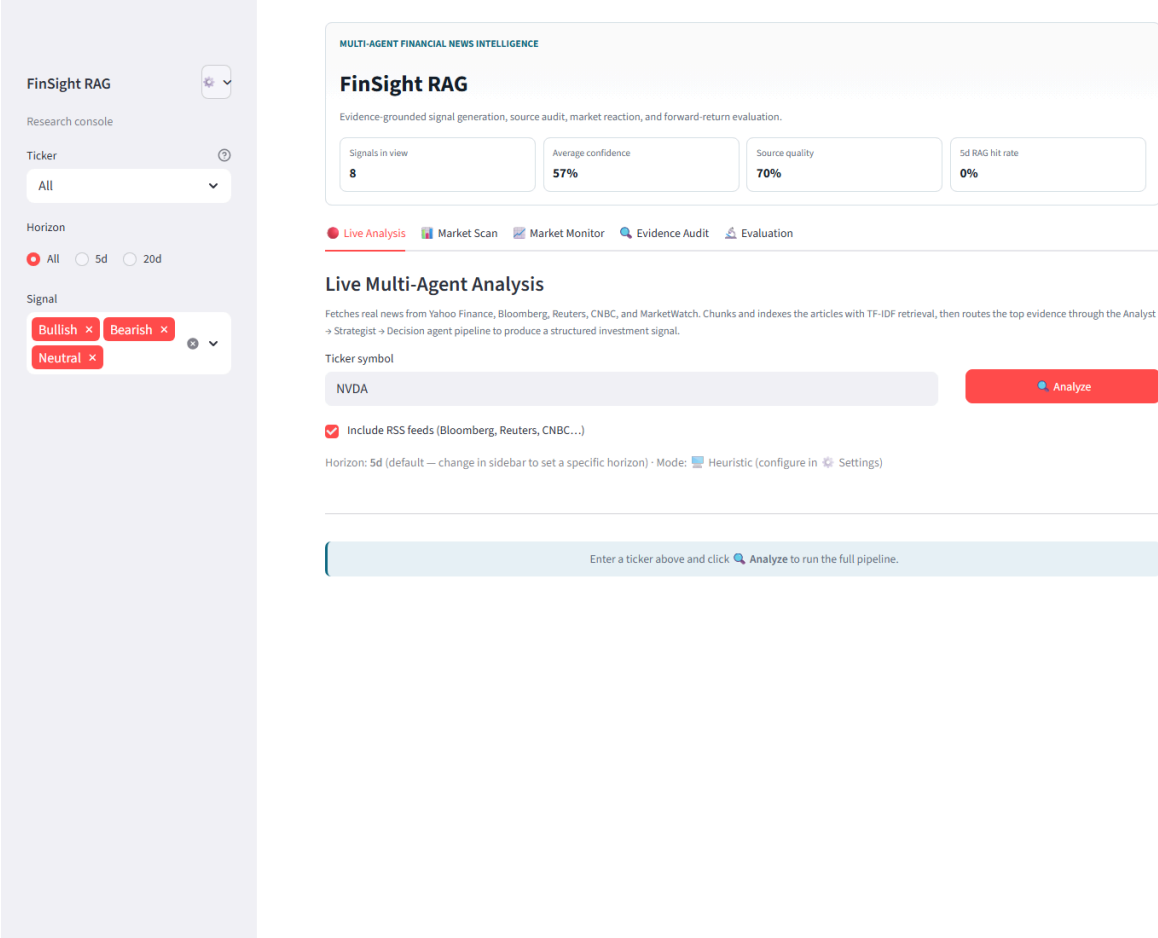
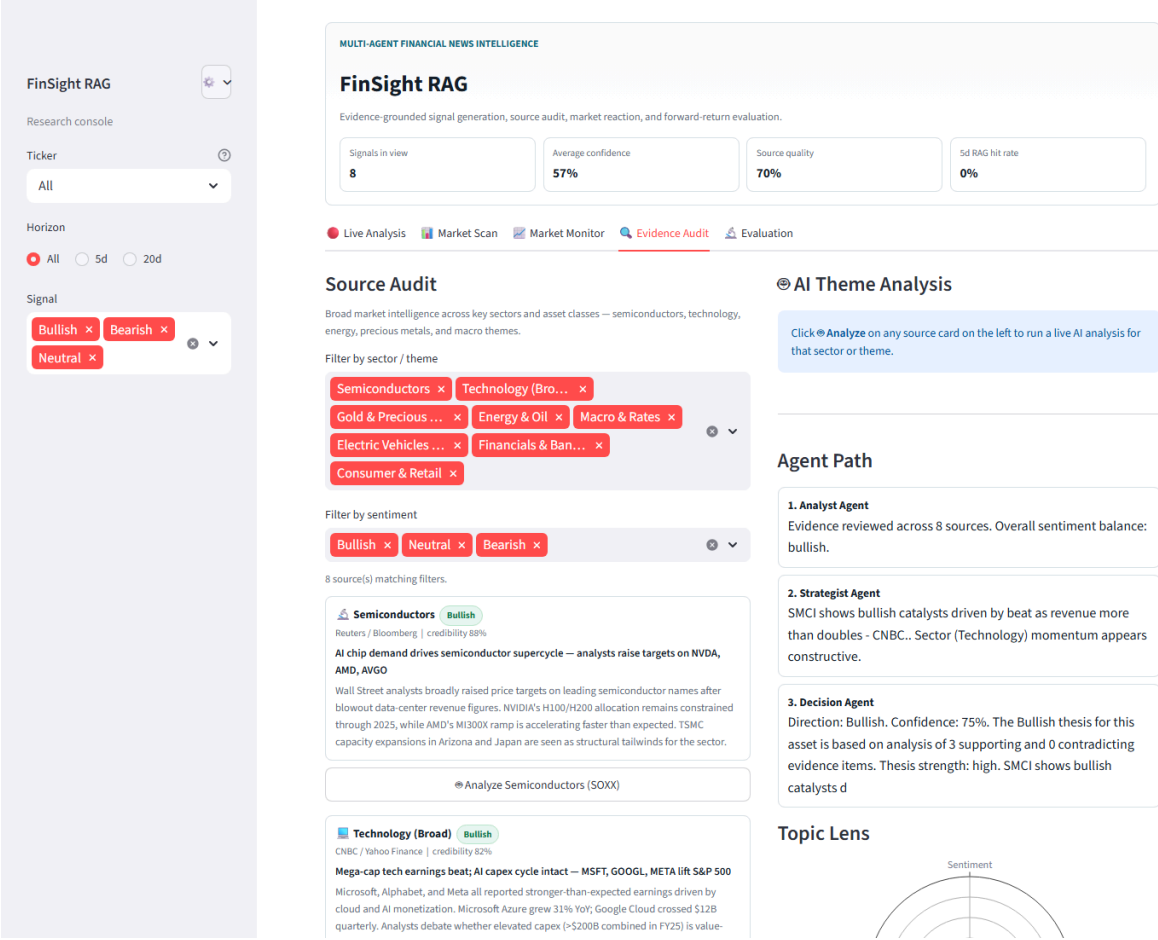


Figure 2. Live Analysis tab for running ticker-level analysis through the FinSight pipeline.



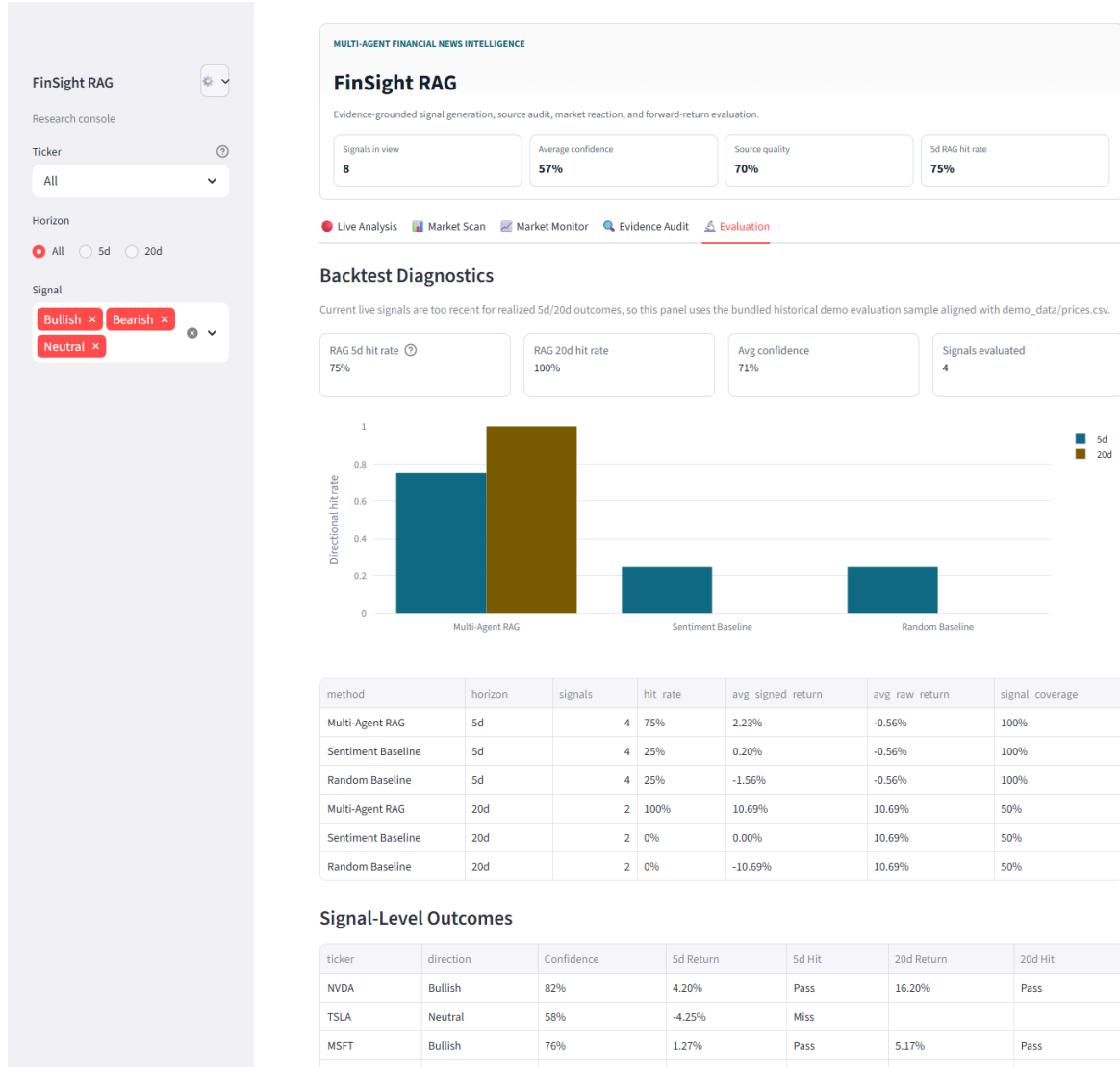


Figure 4. Evaluation Lab view using the historical demo evaluation sample with computed 5d/20d metrics.

The repository also includes a RAG quality testing script (`test_rag_quality.py`) that evaluates Recall@K, Precision@K, MRR, context relevance, completeness, correctness, groundedness, and coherence across a nine-ticker test set.

4. Results and Analysis

At the current stage, confirmed results include implementation-level validation, a small historical demo evaluation sample, and a RAG quality test. This section separates the results into quantitative evaluation and qualitative case analysis. The quantitative evaluation reports computed metrics from the bundled historical sample and the RAG quality script, while the qualitative analysis uses representative cases to explain what those metrics mean in practice.

4.1 Quantitative Evaluation

Table 2 reports computed metrics for the historical demo evaluation sample. This sample is aligned with `demo_data/prices.csv` and is used for report/demo evaluation while the newest live signals are still waiting for realized forward-return windows.

Metric	Value
Number of generated signals	4 historical demo-evaluation signals
5-day directional hit rate	75% for Multi-Agent RAG
20-day directional hit rate	100% for Multi-Agent RAG on the 2 signals with 20d coverage
Average signed return (5d)	2.23%
Average raw return (5d)	-0.56%
Random baseline (5-day hit rate)	25%
Sentiment baseline (5-day hit rate)	25%
Signal coverage rate	100% for 5d; 50% for 20d

Table 2. Historical demo evaluation metrics computed from bundled sample signals and price data.

These demo-evaluation numbers provide an initial quantitative comparison. On the historical sample, the Multi-Agent RAG pipeline achieves a 75% 5-day directional hit rate, compared with 25% for both the keyword-sentiment and random baselines. For the 20-day horizon, Multi-Agent RAG reaches 100% directional hit rate on the two signals with available 20-day coverage, while the baselines do not produce correct 20-day directional predictions in this sample. The average signed 5-day return is positive at 2.23%, although the small sample size means these results should be interpreted as early evidence rather than a statistically strong conclusion. A larger live-data run should be added once the newest signals have realized forward returns.

Figure 4 visualizes the same forward-return comparison in the Evaluation Lab. To complement these market-facing metrics, Table 3 reports an internal RAG quality test across nine tickers. This second quantitative check does not measure trading performance directly; instead, it evaluates whether the system retrieves relevant evidence and produces complete, grounded signal packets.

Metric	Value
Tickers tested	9
Overall pass rate	9/9
Retrieval pass rate	9/9
Generation pass rate	9/9

Average retrieval score	84.4 / 100
Average Recall@K	100.0 / 100
Average Precision@K	55.4 / 100
Average MRR	100.0 / 100
Average context relevance	55.4 / 100
Average generation score	78.8 / 100
Average groundedness	28.9 / 100
Output labels	1 correct, 8 partial

Table 3. RAG quality test results from `test_rag_quality.py` on nine tickers.

The results suggest that the pipeline is functionally reliable: all nine tickers passed retrieval and generation checks, with an average retrieval score of 84.4/100 and Recall@K of 100.0/100. The main weakness is grounding quality: average groundedness is 28.9/100 and 8 of 9 outputs are labeled partial. This means the system can consistently retrieve and structure evidence, but final outputs still need stronger citation grounding and denser evidence support before they should be treated as final analysis.

4.2 Qualitative Case Analysis

To connect these aggregate quality metrics with the system's actual signal behavior, we also inspect two representative historical cases. NVDA shows a successful signal: it was evaluated as Bullish with 82% confidence, and the historical demo sample produced a +4.20% 5-day return and a +16.20% 20-day return, making the signal directionally correct at both horizons.

By contrast, TSLA shows a limitation of the current decision logic. The system evaluated TSLA as Neutral with 58% confidence, but the 5-day realized return was -4.25%, so the neutral signal missed the short-horizon move. This suggests that cautious or neutral outputs can underreact when the available evidence points to a sharper directional move.

The qualitative case analysis keeps four guiding questions in view: (1) Does multi-agent RAG output exceed keyword-sentiment and random baselines on directional hit rate? (2) Are confidence scores calibrated — do high-confidence signals achieve higher hit rates? (3) Does citation grounding improve interpretability, and do higher source-quality scores correlate with better accuracy? (4) What failure modes are most common — retrieval gaps, weak evidence, agent disagreement, or market noise that no public-news signal could have anticipated?

The current implementation partially addresses these questions: the historical demo sample gives an initial baseline comparison, the case studies show early evidence about confidence and failure modes, and the RAG quality test directly measures retrieval relevance and groundedness.

However, the sample is still too small to make strong statistical claims about calibration or the relationship between source quality and market accuracy.

5. Limitations

Final live-data evaluation over a larger sample is not yet complete. The completed numbers above come from the historical demo sample and should be replaced or supplemented after the final run.

The current ticker set (NVDA, TSLA, MSFT, AMD) is small and concentrated in large-cap technology. Generalization to other sectors is untested.

The retrieval layer uses TF-IDF; dense vector retrieval would likely improve semantic recall but is not yet implemented.

RAG quality testing also shows that groundedness remains limited: the system retrieves and structures evidence reliably, but many generated packets are still only partially grounded in the retrieved context.

The sentiment score uses keyword/rule-based heuristics. A domain-adapted model would produce more accurate sentiment estimates.

Short-horizon returns (5-day) are noisy and sensitive to market-wide movements unrelated to the individual news signal.

The EMH [4] implies a theoretical upper bound on the extractable signal from public news; persistent excess returns from public information alone should not be expected.

The LangGraph workflow requires dependency cleanup and environment setup before the final run; heuristic mode is used as a fallback.

6. Future Work

Replace TF-IDF retrieval with a dense vector index (FAISS or Chroma) for improved semantic recall on paraphrase-heavy financial text.

Integrate a domain-adapted sentiment model such as FinBERT [6] to replace the current keyword/rule-based scorer and improve the sentiment baseline quality.

Expand the ticker universe beyond large-cap technology to test generalization across sectors and market caps.

Add a longer-horizon evaluation window (60-day) to assess whether signals have more durable predictive content.

Explore memory-augmented agents that reference prior signals for improved confidence calibration over time.

Investigate retrieval quality metrics (precision@k, NDCG) as leading indicators of downstream signal accuracy.

7. Conclusion

FinSight RAG provides a structured, testable framework for converting financial news into evidence-grounded investment hypotheses. The system combines a credibility-weighted RAG pipeline with a three-agent reasoning workflow that separates evidence extraction, thesis formation, and decision-making into auditable steps. The structured signal packet captures the reasoning context across a defined set of fields, and the evaluation modules provide concrete ways to test both market-facing signal outcomes and internal RAG quality. A larger live-data evaluation and final demo observations should be added after the final run.

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474. [arXiv:2005.11401](https://arxiv.org/abs/2005.11401).
- [2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR 2023)*. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629).
- [3] Wu, S., Irsoy, O., Lu, S., Dabrowski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., and Mann, G. (2023). BloombergGPT: A Large Language Model for Finance. [arXiv:2303.17564](https://arxiv.org/abs/2303.17564).
- [4] Fama, E. F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, 25(2), 383–417. <https://doi.org/10.2307/2325486>.

[5] Loughran, T., and McDonald, B. (2011). When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *The Journal of Finance*, 66(1), 35–65. <https://doi.org/10.1111/j.1540-6261.2010.01625.x>.

[6] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv:1908.10063.